

Wykorzystanie grafowych sieci neuronowych w przewidywaniu animacji szkieletowych postaci 3D

Autor - Patryk Czuchnowski

Promotor - prof. dr hab. inż. Krzysztof Boryczko

Promotor pomocniczy - mgr inż. Mateusz Pawłowicz



Przypomnienie koncepcji rozwiązania

- Model AI operujący na szkieletach który na podstawie X ostatnich klatek animacji jest w stanie wyekstrapolować Y kolejnych
- Innymi słowami przewiduje dalszą część animacji
- Oparty na grafowych sieciach neuronowych
- Model powinien działać na strukturach szkieletów humanoidalnych różniących się zarówno topologicznie jak i morfologicznie



Stack technologiczny

- Python
- Torch geometric
 - Główna biblioteka do pracy nad sieciami grafowymi
- Pymotion
 - Przydatna biblioteka do pracy nad plikami BVH, między innymi pozwala na ich wczytywanie i zapisywanie, przeliczanie kątów rotacji na różne reprezentacje, kinematykę prostą



Architektura modelu

- Model GNN oparty o warstwy GAT
 - Naturalne wsparcie różnych szkieletów
- Wejście: X klatek kontekstu
- Wyjście: Y kolejnych klatek ruchu
- Architektura płaska zamiast Encoder-Decoder
 - Mniejsza liczba parametrów
 - Szybszy trening
 - Po analizie przeprowadzonych eksperymentów uznałem, że nie ma potrzeby stosować architektury Encoder-Decoder
- Połączenia rezydualne



Format danych

- Na wejście modelu podaję kolejne globalne rotacje kości oraz pozycje roota
 - Globalne rotacje dają znacznie lepsze wyniki niż lokalne - lepsza generalizacja
- Rotacje reprezentowane w 6D, pozycje w przestrzeni euklidesowej 3D znormalizowane względem pierwszej klatki
- Czas jest spłaszczony - konkatenuję dane opisujące kolejne klatki w jeden wektor wejściowy
 - Wady: zmiana liczby klatek kontekstu lub klatek generowanych wymaga ponownego treningu modelu
 - Zalety: szybszy trening, znacznie prostsza architektura
- Dane wyjściowe są w takim samym formacie jak wejściowe tylko różni się liczba klatek



Przewidywanie pozycji roota

- Częste źródło problemów, ponieważ pozycje przewidujemy tylko dla jednej kości (roota), oraz jest to zadanie znacznie różniące się od przewidywania rotacji
- Ja zastosowałem architekturę z dwiema głowicami wyjściowymi
 - Wspólny korzeń zawiera warstwy GAT
 - Głowica 1 przewiduje rotacje stawów - na końcu warstwa GAT
 - Głowica 2 przewiduje pozycje roota - na końcu graph mean pooling + MLP



Zbiory danych

- Zbiory zawierają dane z motion capture w formacie BVH
- LAFAN1
- ACCAD
- SFU
- TotalCapture
- Każdy zbiór danych posiada inny szkielet
- Dzięki zastosowaniu sieci grafowych do wsparcia wielu szkieletów wystarczyły niewielkie modyfikacje modelu



Trening

- Adaptacyjny learning rate - ReduceLROnPlateau
- Early stopping
- Mixed precision training
 - Trening w FP16/FP32
 - Mniejsze zużycie pamięci GPU, szybszy trening na CUDA
- Gradient Clipping
 - Zapobiega eksplozji gradientów
- Dropout
 - Technika regularyzacji, zapobiega przeuczeniu



Funkcja straty

$$L = \lambda_{rot}L_{rot} + \lambda_{pos}L_{pos} + \lambda_{smooth}L_{smooth} + \lambda_{root}L_{root}$$

- Strata rotacji
 - Różnica między predykcją modelu a ground truth
 - Zastosowany Geodesic Loss - mierzy rzeczywistą odległość między rotacjami
- Strata pozycji
 - Obliczana na wyjściu modelu po zastosowaniu forward kinematics
 - MSE
 - Jej zastosowanie jest w pewnym sensie redundantne - ale jednak znacznie poprawia wizualnie wyniki



Funkcja straty

$$L = \lambda_{rot}L_{rot} + \lambda_{pos}L_{pos} + \lambda_{smooth}L_{smooth} + \lambda_{root}L_{root}$$

- Strata gładkości
 - Porównuje prędkości ruchu między kolejnymi klatkami
 - Ogranicza nienaturalne duże skoki
 - Poprawia płynność
- Strata pozycji roota
 - MSE pomiędzy predykcją modelu a ground truth



Dziękuję za uwagę